Research Update 5

1. State Before This Work

The last update finished everything Phase 2 owed Phase 3: the noise calibration, the closed review findings, and the noisy JS run. It said the Phase 3 model could start.

This update covers May 31 to June 6, about 20 hours of work. The plan was to start Phase 3 modeling. Before writing any model code, I audited the real training data. The audit found two problems that would have broken the model, so I fixed those first, then built the full Phase 3 training harness with a TKAN model, deployed it to Great Lakes, and ran it. The first runs hit a memory error and a speed limit, and both were fixed.

2. Data Audit

I checked the actual training table, not the summaries. Most of it was fine. The row counts add up (18,669 clean plus 15,050 noisy is 33,719), the last update's tier rates reproduce exactly, and b/blim equals b_mm over blim to 1e-13 with no duplicate keys and no label disagreements.

Two problems came out of it.

First, mirror leakage. A clean dataset and its noisy mirrors are the same cuts with added noise, so they share all their operating points. The fixed dev set held datasets 1 and 91, and 91 is the noisy mirror of 1. A naive split would put the same cuts in both train and test and report a score that is too good.

Second, 700 bad labels. 700 rows, 2.1 percent of the data, sit at low spindle speed with a negative blim, which is physically impossible, and a missing b/blim, yet they are stored with a chatter label of 1. That label cannot be derived from the data. They come from the same blim problem that has been open since Phase 1.

The audit also showed the raw noise score is too skewed to use as a weight, with a max of 190, and that 560 rows have no noise score at all.

3. Fixing the Data

Both problems were fixed with a new data-prep package, built test-first, adding 60 tests for 128 in total.

It does four things. It groups the cross-validation folds so a dataset and its mirrors always stay together, which stops the leakage. It marks and drops the bad rows. It turns the skewed noise score into a clean sample weight. And it fills in six features that were all missing.

One thing nearly broke the fold logic. Different experiments reuse the same speed and depth grid, so unrelated datasets share operating points without being mirrors. The first

leakage check flagged 64 percent of clean points as leaks. I fixed it to check only true mirror groups.

The result is the production table: 33,719 rows, 1,260 dropped (700 bad-label plus 560 no-metrics), 32,459 usable. The sample weight has a mean of 1.0 and a max of 1.62, so the skew is gone. The folds are leakage-verified, and datasets 1, 61, and 91 now all land in the same fold.

4. The Phase 3 Model

With a clean table, I built the Phase 3 training harness in PyTorch, with a Temporal KAN (TKAN) as the first model and an MLP as the baseline to beat.

The model has two heads. The severity head trains now: it predicts b/blim and the chatter label together, weighted by the noise score. The trajectory head is built but gated until it is accurate enough; it rolls the signal forward using a fixed tooth-pass delay that does not need the blocked physics constants. The physics head is a stub, still blocked by the missing Altintas-Budak constants. The KAN layer is built in-house, so there is no outside dependency to install on the cluster.

The build ran in test-driven waves, each fully passing before the next started: the data layer, the KAN layer, the losses, the model, then the trainer. Interpretability was built in too: KAN edge plots, integrated gradients, SHAP, and counterfactuals limited to the two settings an operator can change, speed and depth. The local test suite reached 287 passing.

5. First Great Lakes Run

I packaged the harness for the cluster with self-contained job scripts and an end-to-end test, then deployed it. The install worked, a test subset passed, and the data table rebuilt on the cluster with the same counts as local. I launched both models, 5 folds each.

Two failures followed, both traced to their root cause.

First, an out-of-memory kill. Both models died at 32 GB about four minutes in, in the same way, which pointed at the shared data layer, not the model. It was storing data at double precision, loading every training file twice, and making a full extra copy to compute normalization. The fix uses single precision, loads each file once, and computes normalization in a streaming pass. Peak memory dropped three to four times. Three tests lock the fix.

After that, the MLP baseline finished all 5 folds in about three and a half hours, with a leakage-safe chatter AUC of about 0.87, the first real Phase 3 number. The TKAN did not finish; it hit the time limit. Its slow forward pass on a CPU-only pipeline was the cause, so I added GPU support, sped up data loading, and added a GPU job script. I also found and fixed a wrong accuracy metric while reading the results. The suite is now 295 passing. The TKAN has to be rerun on GPU.

6. Where This Stands

Phase 2 was done before this week. This week did the work that turns a finished pipeline into a model you can trust: a data audit, two problems fixed, and the full Phase 3 harness built, tested, deployed, and run. The MLP baseline ran end to end on real data with leakage-safe cross-validation, which proves the harness works on the cluster. The severity head trains now, the trajectory head is built and gated, and the physics head waits on the same constants as always. What is left is to get the TKAN through a full GPU run and compare it to the baseline.

7. Summary

Last update said Phase 3 could start. This update started it, after first checking the data was sound. The audit found mirror leakage and 700 bad labels, and both were fixed with a leakage-safe data-prep package, giving a clean table of 32,459 rows over 5 verified folds. The full Phase 3 harness was built in PyTorch with a TKAN model and an MLP baseline, interpretability built in, at 295 tests passing. It deployed to Great Lakes and ran; a memory error and a speed limit were both fixed at the root. The MLP baseline gave a first leakage-safe chatter AUC of about 0.87. The TKAN still needs a full GPU run. Phase 3 is now underway on data and code that can be trusted.

8. Time Spent

This update covers May 31 to June 6, about 20 hours of work.

Work	Hours
Data audit on the real training table	2.0
Leakage-safe data-prep package, with tests	3.0
TKAN design and plan	1.0
KAN layer, temporal cell, and encoder	2.5
Data layer, losses, and metrics	2.0
Heads, MLP baseline, and model assembly	1.5
Trainer, command-line tool, and config	1.5
Interpretability, job scripts, and end-to-end test	2.0
Cluster deploy and job launch	1.0
Memory fix in the data layer	1.5
GPU support, speed-ups, and metrics fix	1.5
Docs and this update	0.5
Total hours	**20.0**

The two biggest blocks were the data-prep package and the model build. The audit before them took little time but is why the rest can be trusted.